Scripts PowerShell

Gestion des erreurs

OBJECTIFS

- Gérer les erreurs de scripts
- Différencier les types d'erreurs

Rester dans voter répertoire personnel P:\Documents (utilisez la commande set-location lequel vous allez éditer et tester vos scripts.

RAPPELS

Voyons le fonctionnement général des erreurs avec PowerShell afin d'apprendre à bien gérer les erreurs dans les scripts. Ceci va passer par l'utilisation des blocs d'instructions **Try-Catch-Finally** afin d'indiquer à notre script ce qu'il doit faire lorsqu'une erreur est rencontrée.
Voyons également comment différencier une erreur fatale d'une erreur non fatale, et découvrons le paramètre **ErrorAction** et la variable de préférence **ErrorActionPreference**.

FONCTIONNEMENT DES ERREURS

Quand vous allez écrire vos scripts PowerShell, vous serez obligatoirement confrontés à des erreurs. Même un code parfaitement écrit et qui respecte les bonnes pratiques est susceptible de générer des erreurs : une **valeur inattendue** pour un paramètre de cmdlet, une **commande mal écrite**, un **problème de droits** sur un ordinateur, une opération non autorisée, etc... Les situations lors desquelles nous pouvons rencontrer une erreur sont nombreuses... Et, nous devons être capables d'interpréter ces erreurs, mais également de les gérer.

Ici nous allons évoquer le fonctionnement des erreurs avec PowerShell, car ce sera essentiel pour la suite puisque dans le prochain chapitre, nous verrons comment gérer les erreurs avec "**Try-Catch et Finally**".
Au-delà de la variable automatique "**$?**" qui permet de savoir si la dernière commande s'est correctement exécutée ou non (un booléen est retourné), PowerShell met à notre disposition la variable "**$Error**", ainsi que le cmdlet "**Get-Error**".

Les erreurs fatales et non fatales

Dans PowerShell, il existe deux types principaux d'erreurs : les **erreurs fatales** et les **erreurs non fatales**, que nous pouvons aussi appeler les erreurs bloquantes et les erreurs non bloquantes. En anglais, deux termes sont associés : *terminating error* et *non-terminating error*.

Les erreurs fatales sont des erreurs graves qui arrêtent immédiatement l'exécution du script. Elles peuvent se produire dans différentes situations et un cmdlet ne retournera pas systématiquement une erreur fatale. Cette erreur peut se produire si un problème est rencontré et qu'il bloque la suite de l'exécution.
À l'inverse, et vous devez vous en douter, les erreurs non fatales n'arrêtent pas l'exécution du script. Elles signalent simplement qu'une erreur s'est produite, mais le script continue à s'exécuter. La gestion de ces deux types d'erreurs est essentielle dans vos scripts.

La variable $Error

Lorsque vous travaillez avec PowerShell, il est important de comprendre comment utiliser et interagir avec la variable "**$Error**". Cette variable gérée par PowerShell joue un rôle crucial puisque nous allons l'utiliser pour **récupérer les dernières erreurs**, ce qui sera fort utile en phase de débogage. **Elle contient les erreurs fatales et non fatales**.

Tout d'abord, sachez que **la variable "$Error" est une variable automatique** qui contient **un tableau des dernières erreurs** qui se sont produites lors de l'exécution d'un script ou d'une commande. **L'erreur la plus récente est toujours à l'index 0 du tableau**. À chaque fois qu'une erreur que l'on appelle aussi exception est générée, celle-ci est ajoutée au tableau "**$Error**".

Son contenu est éphémère puisqu'elle est nettoyée à la fin de chaque session PowerShell. Autrement dit, si vous fermez la console PowerShell et que vous la rouvrez, "**$Error**" sera vide.

En pratique, la variable "**$Error**" peut être utilisée pour **obtenir des informations détaillées sur les erreurs**. Vous pouvez afficher toutes les erreurs en saisissant tout simplement "**$Error**" dans la console. De plus, vous pouvez accéder à une erreur spécifique en spécifiant son numéro d'index.

Exécutez les deux commandes ci-dessous afin de générer deux erreurs :

```
Get-Contenu -Path "C:\temp\fichier.txt"
Get-Content -Path "C:\temp\fichier.txt"
```

Dans le premier cas, le nom du cmdlet est incorrect tandis que dans le second cas, le fichier est introuvable. Nous pouvons consulter la variable "**$Error**" pour afficher ces deux erreurs :

```
$Error
```

```
PS C:\>
PS C:\> $Error
Get-Content : Impossible de trouver le chemin d'accès « C:\temp\fichier.txt », car il n'existe pas.
Au caractère Ligne:1 : 1
+ Get-Content -Path "C:\temp\fichier.txt"
+ ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (C:\temp\fichier.txt:String) [Get-Content], ItemNotFoundException
    + FullyQualifiedErrorId : PathNotFound,Microsoft.PowerShell.Commands.GetContentCommand

Get-Contenu : Le terme «Get-Contenu» n'est pas reconnu comme nom d'applet de commande, fonction, fichier de script ou
programme exécutable. Vérifiez l'orthographe du nom, ou si un chemin d'accès existe, vérifiez que le chemin d'accès
est correct et réessayez.
Au caractère Ligne:1 : 1
+ Get-Contenu -Path "C:\temp\fichier.txt"
+ ~~~~~~~~~~~~
    + CategoryInfo          : ObjectNotFound: (Get-Contenu:String) [], CommandNotFoundException
    + FullyQualifiedErrorId : CommandNotFoundException

PS C:\>
```

Puis, si nous souhaitons uniquement **récupérer l'erreur la plus récente** (ou une erreur spécifique vis-à-vis de son index) :

```
$Error[0]
```

Plutôt que de récupérer tout le message d'erreur, nous pouvons faire en sorte de récupérer certaines informations précises.

- Récupérer uniquement le texte correspondant à l'erreur :

```
$Error[0].Exception.Message
```

Ici, ce sera : "Impossible de trouver le chemin d'accès « C:\temp\fichier.txt », car il n'existe pas.".

- Récupérer le numéro de ligne dans le script :

```
$Error[0].InvocationInfo.ScriptLineNumber
```

Ici, ce sera "1" car il s'agit d'une commande exécutée dans la console, et non d'un script.

- Récupérer le type d'erreur généré :

```
$Error[0].Exception.GetType().FullName
```

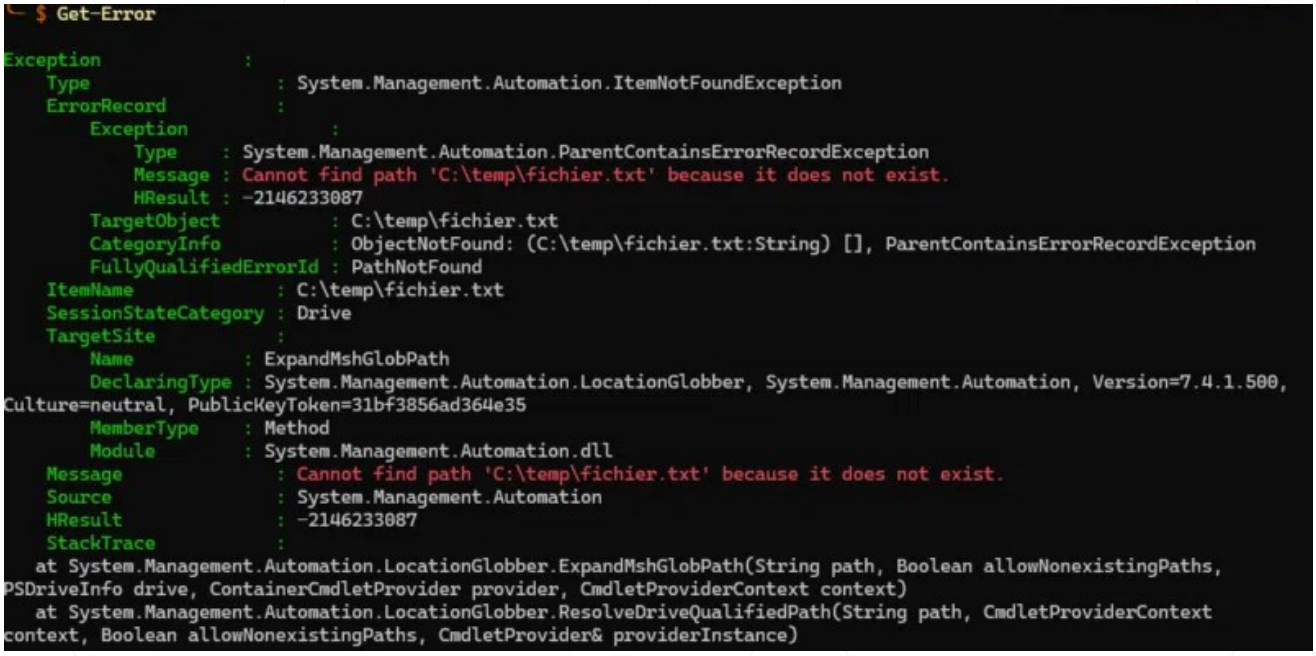
Ici, la valeur retournée est : "*System.Management.Automation.ItemNotFoundException*".
À chaque fois, selon l'information que nous souhaitons récupérer, nous allons lire une propriété différente.

Le cmdlet Get-Error

Depuis **PowerShell 7**, nous avons la possibilité d'utiliser le cmdlet "**Get-Error**" pour obtenir des informations sur les dernières erreurs rencontrées dans la session actuelle. Le cmdlet "**Get-Error**" permet d'obtenir un **objet "PSExtendedError"** et elle est alimentée à partir des informations contenues dans "**$Error**".

```
Get-Error
```

Le résultat retourné par "**Get-Error**" est mis en forme, en comparaison de la lecture directe de "**$Error**".



Ce qui est pratique avec "**Get-Error**", c'est le paramètre "**-Newest**" car il permet d'obtenir les X plus récentes erreurs. Par exemple, pour **obtenir les trois erreurs les plus récentes**, nous devons simplement exécuter cette commande :

```
Get-Error -Newest 3
```

La variable $ErrorActionPreference

"**$ErrorActionPreference**" est une variable de préférence intégrée dans PowerShell qui détermine comment il doit gérer les erreurs non bloquantes. Plusieurs valeurs sont valides et par défaut la valeur est "**Continue**".

Les valeurs suivantes sont acceptées : **SilentlyContinue**, **Stop**, **Continue**, **Inquire**, **Ignore**, **Suspend** et **Break**. Il est à noter que la valeur "**Break**" n'est pas prise en charge par Windows PowerShell, et que les valeurs "**Ignore**" et "**Suspend**" sont prises en charge uniquement par le paramètre "**-ErrorAction**" que nous allons étudier par la suite.

Valeur	Description
SilentlyContinue	Les erreurs sont supprimées et le script continue son exécution.
Stop	L'exécution du script est arrêtée dès qu'une erreur survient.
Continue	Les erreurs sont affichées dans la console, mais le script continue son exécution. **Valeur par défaut**.
Inquire	PowerShell demande à l'utilisateur s'il veut continuer chaque fois qu'une erreur se produit.
Ignore	L'erreur retournée par la commande est ignorée et n'est même pas ajoutée à la variable $Error.
Suspend	Pour les workflows PowerShell uniquement, afin de le suspendre en cas d'erreur pour permettre le débogage.
Break	Quand une erreur se produit, PowerShell active le débogueur sur l'instruction en cours d'exécution.

Pour définir une autre valeur, il convient d'affecter une nouvelle valeur à la variable :

```
$ErrorActionPreference = "stop"
```

La valeur de la variable "**$ErrorActionPreference**" peut être modifiée dans le profil PowerShell pour être persistante ou directement dans une console PowerShell pour s'appliquer uniquement le temps que la console est ouverte, ou en début de script. Néanmoins, il est plutôt recommandé d'utiliser le paramètre "**-ErrorAction**".

Le paramètre ErrorAction

Le paramètre "**-ErrorAction**" est un **paramètre commun** que vous pouvez **utiliser avec n'importe quelle cmdlet dans PowerShell**, y compris avec une fonction que vous avez développée si celle-ci bénéficie du paramètre CmdletBinding().

Ce paramètre vous permet de spécifier le comportement de la commande exécutée si une erreur non bloquante se produit lors de l'exécution. Les valeurs que vous pouvez attribuer à "**-ErrorAction**" sont les mêmes que celles de la variable "**$ErrorActionPreference**", à savoir : "**SilentlyContinue**", **Stop**, **Continue**, **Inquire**, **Ignore**, **Suspend** et **Break**.

Par exemple, si vous utilisez "**Stop**" comme valeur, une erreur fatale sera générée et l'exécution du script sera stoppée (sauf si un bloc d'instruction "**Try-Catch**" est utilisé pour capturer l'erreur). Si vous utilisez "**SilentlyContinue**", le cmdlet ignorera l'erreur et le script poursuivra son exécution. Ce paramètre est très utile pour **gérer les erreurs directement au niveau des cmdlets**, de façon individuelle et indépendante, au sein d'un script PowerShell.
Voici comment utiliser ce paramètre :

```
Get-Content -Path "C:\temp\fichier.txt" -ErrorAction Stop
```

GÉRER LES ERREURS AVEC TRY-CATCH-FINALLY

Voyons comment utiliser les blocs d'instructions **Try**, **Catch** et **Finally** avec PowerShell, car ils vont jouer un rôle clé dans la gestion des erreurs dans un script PowerShell.

En effet, ce bloc d'instruction est très pratique car il permet d'essayer d'exécuter le code contenu dans le bloc "**Try**" et si ce code retourne une erreur (et uniquement dans ce cas), on exécute le code contenu dans le bloc "**Catch**". Ceci est possible parce que le "**Catch**" sera parvenu à capturer l'erreur retournée par le bloc "Try". En complément, il y a le bloc "**Finally**" qui est optionnel, car il est toujours exécuté. Grâce à un bloc d'instruction "**Try-Catch**", vous allez pouvoir gérer les erreurs proprement dans vos scripts PowerShell.

PowerShell et la syntaxe de Try, Catch et Finally

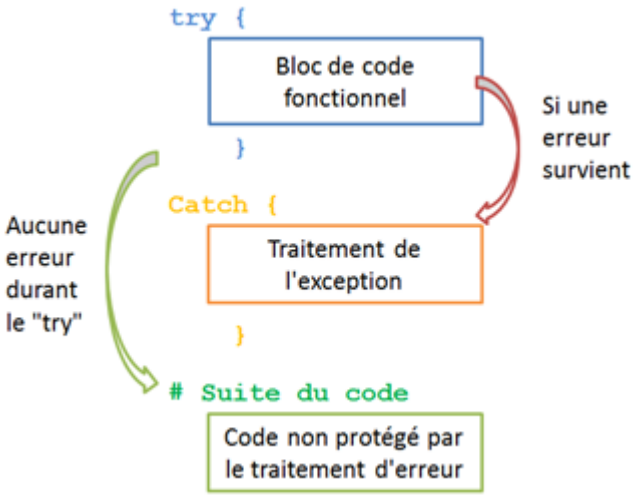
Comme nous allons le voir au travers de différents exemples, le couple d'instructions "**Try-Catch**" est très pratique et personnellement, je l'utilise énormément dans mes scripts. En revanche, le dernier bloc "**Finally**" n'est pas très souvent utilisé à cause de son fonctionnement : il est systématiquement exécuté, qu'il y ait une erreur ou non. Voici la syntaxe d'un bloc Try-Catch-Finally :

```
Try {
   <code à exécuter, et qui potentiellement, peut générer une erreur>
}
Catch {
   <code à exécuter lorsqu'une erreur se produit dans le Try>
}
Finally {
   # Optionnel
   <code à exécuter dans tous les cas, peu importe le résultat de Try>
}
```

Le bloc "Finally" est facultatif et le code contenu dans ce bloc sera toujours exécuté, quel que soit le résultat du bloc "**Try**". Autrement dit, que le code soit dans l'instruction Finally ou à la suite du script après le "**Catch**", cela ne change rien puisqu'il sera exécuté dans les deux cas. J'ai envie de dire qu'il est surtout présent pour produire un code propre et pour conserver une certaine logique vis-à-vis de l'instruction présente dans le "**Try-Catch**".
Sans l'instruction "**Finally**", la syntaxe est la suivante :

```
try
{
   <code à exécuter, et qui potentiellement, peut générer une erreur>
}
catch
{
   <code à exécuter lorsqu'une erreur se produit dans le Try>
}
```

Pour résumer, voici la logique de fonctionnement générale :



Avant de passer aux exemples, sachez qu'il est possible d'**intégrer plusieurs instructions Catch pour traiter les erreurs au cas par cas**, comme nous le verrons dans la prochaine partie de cet article. **Par contre, les blocs "Try" et "Finally" sont forcément uniques.**

Exemples Try, Catch et Finally avec PowerShell

Au travers des exemples ci-dessous, vous allez **apprendre à manipuler les blocs d'instructions "Try-Catch-Finally"** en PowerShell. Ces différents exemples vont vous permettre de bien comprendre le fonctionnement de "**Try-Catch**".

Try-Catch : un premier exemple basique

Nous allons chercher à récupérer le contenu du fichier "hosts" de notre machine Windows et à le stocker dans une variable. Pour rappel, voici le chemin complet vers ce fichier : "C:\Windows\System32\drivers\etc\hosts". En cas d'erreur, nous afficherons seulement un message pour dire que le fichier est introuvable.

- Script "TryCatch1.ps1" :

```
$Fichier = "C:\Windows\System32\drivers\etc\hosts"
try
{
   $ContenuFichier = Get-Content $Fichier
   Write-Host -ForegroundColor Green "Contenu du fichier récupéré ($Fichier)"
}
catch
{
   Write-Host "Attention, le fichier $Fichier est introuvable !" -ForegroundColor Red
}
```

Si vous exécutez ce script, vous allez obtenir le résultat suivant :

```
PS C:\TEMP\POWERSHELL> .\TryCatch1.ps1
Contenu du fichier récupéré (C:\Windows\System32\drivers\etc\hosts)
```

C'est normal puisque le fichier existe donc le cmdlet "**Get-Content**" ne retourne pas d'erreur. Cela signifie que le code présent dans le bloc "**Catch**" ne sera pas exécuté, car "**Try**" n'a pas renvoyé d'erreur.
Maintenant, faisons un essai en prenant un fichier qui n'existe pas. Ce qui donne un second script où nous allons simplement changer le nom du fichier affecté à la variable "$Fichier".

- Script TryCatch2.ps1 :

```
$Fichier = "C:\Windows\System32\drivers\etc\monfichier"
try
{
   $ContenuFichier = Get-Content $Fichier
   Write-Host -ForegroundColor Green "Contenu du fichier récupéré ($Fichier)"
}
catch
{
   Write-Host "Attention, le fichier $Fichier est introuvable !" -ForegroundColor Red
}
```

Si vous exécutez ce script, vous allez obtenir un résultat qui n'est pas forcément celui que vous attendiez... L'erreur associée à "**Get-Content**" va s'afficher dans la console (l'erreur est normale puisque le fichier n'existe pas) et le message "Contenu du fichier récupéré...." s'affiche aussi.
Par contre, aucune trace de notre phrase "Attention, le fichier $Fichier est introuvable !" malgré la présence de l'erreur ! **Le bloc "Catch" ne semble pas s'exécuter**. Mais, pourquoi ?

```
PS C:\TEMP\POWERSHELL> .\TryCatch2.ps1
Get-Content: C:\TEMP\POWERSHELL\TryCatch2.ps1:5
Line |
   5 |      $ContenuFichier = Get-Content $Fichier
     |                        ~~~~~~~~~~~~~~~~~~~~
     | Cannot find path 'C:\Windows\System32\drivers\etc\monfichier' because it does not exist.

Contenu du fichier récupéré (C:\Windows\System32\drivers\etc\monfichier)
PS C:\TEMP\POWERSHELL>
```

Lorsqu'une commande ou un script PowerShell renvoie une erreur, il y a deux types d'erreurs : des erreurs qui stoppent l'exécution (*terminating error*) et les erreurs qui n'empêchent pas la suite du script de s'exécuter (*non-terminating error*).

Un bloc d'instruction "Try-Catch" agit seulement sur les erreurs de type "terminating error", et ce comportement dépend de deux choses :

- Le cmdlet utilisé dans le bloc "**Try**" et du type d'erreur qu'il génère
- La valeur de la variable "**$ErrorActionPreference**" ou du paramètre "**-ErrorAction**" au niveau du cmdlet

Pour capturer toutes les erreurs avec un bloc Try-Catch, **il faut convertir les erreurs non-terminating (que l'on pourrait appeler erreur non fatale) en erreur terminating (fatale)**. Pour cela, nous pouvons agir sur la variable de préférence "**$ErrorActionPreference**" ou le paramètre "**-ErrorAction**" pris en charge au niveau de chaque cmdlet.

```
$ErrorActionPreference
```

La variable "**$ErrorActionPreference**" est définie au sein de votre profil PowerShell et elle définit le comportement à adopter en cas d'erreur. Par défaut, sa valeur est "**Continue**" cela signifie qu'en cas d'erreur, nous poursuivons l'exécution du script. Pour convertir toutes les erreurs non fatales en erreurs fatales, il faudrait définir :

```
$ErrorActionPreference = "stop"
```

Cette valeur peut être modifiée dans le profil PowerShell pour être persistante ou directement dans une console PowerShell pour s'appliquer uniquement le temps que la console est ouverte, ou dans un script. Si c'est dans le profil, c'est dangereux, car cela signifie que le paramètre est propre à l'environnement d'exécution local.

```
-ErrorAction
```

Chaque cmdlet dispose d'un paramètre natif nommé "**-ErrorAction**" qui permet de préciser le comportement à adopter lorsqu'une erreur se produit. Par exemple, pour masquer le message d'erreur dans la console et continuer l'exécution du script, on indiquera :

```
-ErrorAction SilentlyContinue
```

Pour convertir les erreurs non fatales en erreur fatale, il faut utiliser la valeur "stop". Sur une commande complète, cela donne :

```
Get-Content $Fichier -ErrorAction Stop
```

De cette façon, **le bloc "Try-Catch" va fonctionner sur "Get-Content" en cas d'erreur** alors qu'il ne fonctionnait pas dans l'exemple précédent. Essayons avec ce troisième bout de code (toujours sur le fichier qui n'existe pas) :

- Script TryCatch3.ps1 :

```
$Fichier = "C:\Windows\System32\drivers\etc\monfichier"
try
{
   $ContenuFichier = Get-Content $Fichier -ErrorAction Stop
   Write-Host -ForegroundColor Green "Contenu du fichier récupéré ($Fichier)"
}
catch
{
   Write-Host "Attention, le fichier $Fichier est introuvable !" -ForegroundColor Red
}
```

Dans le bout de code ci-dessus, vous remarquerez la présence de "**-ErrorAction Stop**" à la fin de la commande "Get-Content". Si j'exécute le script, j'obtiens ceci :

```
PS C:\TEMP\POWERSHELL> .\TryCatch3.ps1
Attention, le fichier C:\Windows\System32\drivers\etc\monfichier est introuvable !
PS C:\TEMP\POWERSHELL>
```

Nous pouvons voir que l'instruction contenue dans le bloc "Catch" s'est bien exécutée ! L'erreur a été capturée et nous avons pu obtenir le résultat attendu !

Try-Catch : afficher le message d'erreur

Dans l'exemple précédent, nous avons pris la décision d'afficher un message d'erreur personnalisé ("Attention, le fichier...."), mais il ne donne pas de détails précis sur l'erreur exacte.

Pour que ce soit plus pertinent, nous allons voir que l'on peut **capturer et afficher le message correspondant à l'erreur générée**. Nous savons que nous pouvons récupérer le dernier message d'erreur en consultant la constante "**$Error**" comme ceci :

```
$Error[0].Exception.Message
```

L'index [0] est important puisque le message d'erreur le plus récent est toujours ajouté en premier, et donc il correspond à l'index 0. Si nous ne précisons pas ce numéro d'index, nous allons afficher tous les messages d'erreurs générés lors de la session en cours ou depuis la dernière remise à zéro de la constante "**$Error**" (ce qui pourrait être fait avec cette méthode : **$Error.Clear()**).

```
$Error.Exception.Message
```

Pour récupérer le message d'erreur dans un bloc d'instructions "**Try-Catch**", nous allons pouvoir utiliser cette méthode en précisant l'index 0, mais nous pouvons aussi utiliser une autre syntaxe grâce à l'objet en cours "**$_**" :

```
$_.Exception.Message
```

Note : une autre alternative consiste à utiliser "**$PSItem.Exception.Message**".
Si nous reprenons le script précédent et que nous modifions le bloc "**Catch**", cela donne le script "TryCatch4.ps1" avec le contenu suivant :

```
$Fichier = "C:\Windows\System32\drivers\etc\monfichier"
try
{
   $ContenuFichier = Get-Content $Fichier -ErrorAction Stop
   Write-Host -ForegroundColor Green "Contenu du fichier récupéré ($Fichier)"
}
catch
{
   Write-Host $_.Exception.Message -ForegroundColor Red
}
```

Une fois que ce script est exécuté, toujours en ciblant un fichier qui n'existe pas, nous obtenons un retour court, mais précis dans la console puisque nous avons le nom précis de l'erreur.

```
Cannot find path 'C:\Windows\System32\drivers\etc\monfichier' because it does not exist.
```

Si nous comparons le script "TryCatch2" (erreur complète) avec le script "TryCatch4" (uniquement le message de l'exception), nous pouvons voir cela fait beaucoup plus propre :

```
PS C:\TEMP\POWERSHELL> .\TryCatch2
Get-Content: C:\TEMP\POWERSHELL\TryCatch2.ps1:5
Line |
   5 |      $ContenuFichier = Get-Content $Fichier
     |                        ~~~~~~~~~~~~~~~~~~~~
     | Cannot find path 'C:\Windows\System32\drivers\etc\monfichier' because it does not exist.

Contenu du fichier récupéré (C:\Windows\System32\drivers\etc\monfichier)
PS C:\TEMP\POWERSHELL>
PS C:\TEMP\POWERSHELL> .\TryCatch4.ps1
Cannot find path 'C:\Windows\System32\drivers\etc\monfichier' because it does not exist.
PS C:\TEMP\POWERSHELL>
```

Try-Catch avec plusieurs Catch pour gérer les différentes erreurs

Une commande PowerShell peut retourner plusieurs erreurs différentes, car la cause de l'erreur n'est pas toujours la même. Si nous prenons l'exemple de la commande New-Item qui permet de créer des éléments (fichiers, dossiers), une erreur peut être générée pour plusieurs raisons :

- Le lecteur cible où l'on souhaite créer le fichier (ou le dossier) n'existe pas
- Le dossier cible où l'on souhaite créer le fichier n'existe pas
- Le fichier que l'on essaie de créer existe déjà
- Etc.

Grâce à un bloc d'instructions "**Try-Catch**", nous allons pouvoir **gérer chaque erreur indépendamment grâce à plusieurs blocs "Catch"**. Par exemple, si le dossier cible n'existe pas, nous pouvons décider de le créer, tandis que si le fichier existe déjà, nous pouvons tenter de créer le fichier avec un nom alternatif.

Pour capturer une erreur spécifique dans un bloc "**Catch**", commençons par récupérer le nom précis de cette erreur. Pour cela, nous allons générer l'erreur. Prenons le cas où le lecteur dans lequel nous voulons créer le fichier n'existe pas. Nous allons exécuter la commande suivante :

```
New-Item Z:\fichier.txt
```

Une erreur s'affiche... mais la console n'affiche pas son nom technique, disons. Pour cela, il faut regarder un peu plus en détail les propriétés disponibles pour cette dernière erreur, grâce à "**Get-Member**". Ce qui donne :

```
$Error[0].Exception | Get-Member
```

Cela va lister toutes les propriétés et méthodes que nous pouvons appliquer sur la commande passée en entrée. En l'occurrence, l'information qui nous intéresse est ailleurs : il s'agit de la **valeur "TypeName" affichée au-dessus du tableau de valeurs**. Dans cet exemple, l'erreur est :

```
System.Management.Automation.DriveNotFoundException
```

```
PS C:\TEMP\POWERSHELL> New-Item Z:\fichier.txt
New-Item: Cannot find drive. A drive with the name 'Z' does not exist.
PS C:\TEMP\POWERSHELL> $Error[0].Exception | Get-Member

   TypeName: System.Management.Automation.DriveNotFoundException

Name                      MemberType Definition
----                      ---------- ----------
Equals                    Method     bool Equals(System.Object obj)
GetBaseException          Method     System.Exception GetBaseException()
GetHashCode               Method     int GetHashCode()
GetObjectData             Method     void GetObjectData(System.Runtime.Serialization.Serializ
GetType                   Method     type GetType()
ToString                  Method     string ToString()
Data                      Property   System.Collections.IDictionary Data {get;}
ErrorRecord               Property   System.Management.Automation.ErrorRecord ErrorRecord {ge
HelpLink                  Property   string HelpLink {get;set;}
HResult                   Property   int HResult {get;set;}
```

Une autre méthode consiste à récupérer le type via "**GetType()**" puis la propriété "**FullName**" associée afin de retourner dans la console uniquement le nom, comme ceci :

```
$Error[0].Exception.GetType().FullName
```

Il va falloir que notre bloc "Catch" capture l'erreur "**System.Management.Automation.DriveNotFoundException**" pour lui réserver un traitement particulier lorsque nous tombons sur un cas où le lecteur n'existe pas. Dans le même esprit, si nous voulons gérer le cas où le dossier cible n'existe pas, il faut gérer l'erreur "**System.IO.DirectoryNotFoundException**".

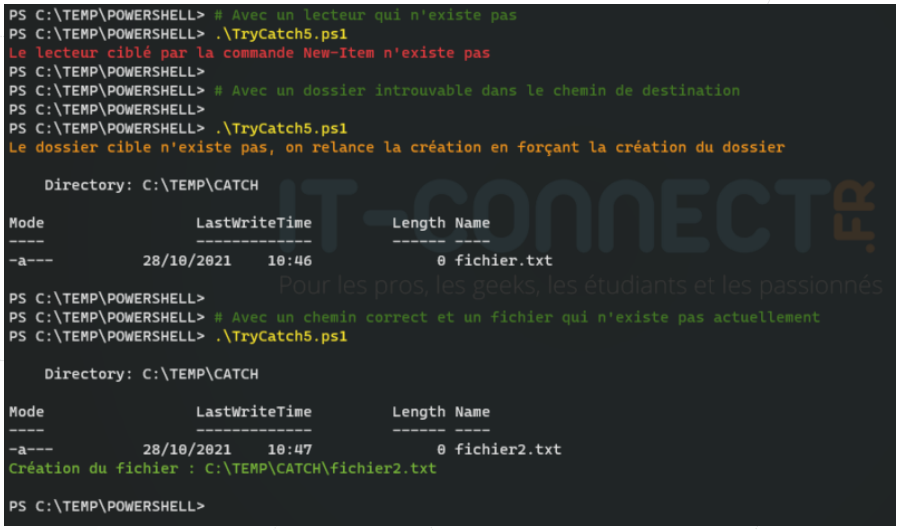
Voici le code du script "TryCatch5.ps1" avec plusieurs blocs catch :

```
$Fichier = "C:\TEMP\CATCH\fichier2.txt"
try
{
   New-Item -ItemType File -Path $Fichier -ErrorAction Stop
   Write-Host -ForegroundColor Green "Création du fichier : $Fichier"
}
catch [System.Management.Automation.DriveNotFoundException]
{
   Write-Host "Le lecteur ciblé par la commande New-Item n'existe pas" -ForegroundColor Red
}
catch [System.IO.DirectoryNotFoundException]
{
   Write-Host "Le dossier cible n'existe pas, on relance la création en forçant la création du dossier" -ForegroundColor DarkYellow
   New-Item -ItemType File -Path $Fichier -Force -ErrorAction Stop
}
catch
{
   Write-Host $_.Exception.Message -ForegroundColor Red
}
```

Dans le bout de code ci-dessus, nous pouvons voir que si l'on veut déclarer plusieurs blocs catch, ce n'est pas plus compliqué qu'avec un seul. Néanmoins :

- Il ne faut pas essayer de gérer deux fois la même erreur
- Il faut ajouter un bloc catch générique à la fin pour gérer toutes les erreurs non traitées spécifiquement, mais ce n'est pas obligatoire si vous souhaitez vous intéresser seulement à une erreur spécifique
- L'espace entre "catch" et la déclaration de l'erreur "[...]" est indispensable d'un point de vue syntaxique

Pour tester ce bout de code, c'est assez simple, il suffit de jouer sur la variable "$Fichier" pour générer les différents types d'erreurs que nous capturons avec nos blocs catch spécifique (lecteur qui n'existe pas et dossier cible qui n'existe pas).
Voici ce que ça donne en image :



Exemple de Try-Catch-Finally

Nous n'avons pas encore utilisé le bloc facultatif "**Finally**" jusqu'ici, alors c'est ce que nous allons faire dans ce quatrième exemple. **Pour rappel, le code au sein du bloc "Finally" sera exécuté dans tous les cas, peu importe si le bloc "Try" retourne une erreur ou non**.

Nous allons récupérer l'état du service dans le bloc "**Try**" et en cas d'erreur, nous l'afficherons via le bloc "**Catch**". Enfin, nous utiliserons le bloc "**Finally**" pour purger le contenu de la constante "***$Error***". Nous aurions pu imaginer autre chose, comme par exemple, démarrer le service en question.
À la suite de notre bloc d'instructions "**Try-Catch-Finally**", nous afficherons le contenu de "**$Error**". Normalement, la variable devrait être vide, car nous allons la purger au sein du bloc "**Finally**". Ce sera l'occasion de confirmer que le code contenu dans le bloc "**Finally**" s'est bien exécuté.
Voici le contenu du script TryCatch6.ps1 :

```
$NomService = "serviceinexistant"
try
{
   $EtatService = Get-Service -Name $NomService -ErrorAction Stop
   Write-Host -ForegroundColor Green "Etat du service correctement récupéré !"
}
catch
{
   Write-Host $_.Exception.Message -ForegroundColor Red
}
finally
{
   $Error.Clear()
}
Write-Host "Contenu des erreurs : $Error"
```

Voici ce que nous obtenons :

```
PS C:\TEMP\POWERSHELL> .\TryCatch6.ps1
Cannot find any service with service name 'serviceinexistant'.
Contenu des erreurs :
PS C:\TEMP\POWERSHELL>
```

Nous pouvons voir que le bloc "**Finally**" s'est bien exécuté puisque la variable "**$Erreur**" ne renvoie rien. Mais, finalement, nous aurions écrit la ligne "**$Error.Clear()**" à la suite du bloc d'instructions "**Try-Catch**", sans utiliser "**Finally**", le résultat aurait été le même.

CONCLUSION

Dans un script, la gestion des erreurs est importante pour anticiper les éventuels problèmes, et pourquoi pas, les traiter et les solutionner à la volée.